

Phase 2: Building AI That Can Act

Module 5: Tool Calling and MCP

Extending what your assistant can reach for

Agenda

- Journey checkpoint & learning objectives
- Tool calling options
- Why MCP (and why not just CLI access)
- Components & capabilities
- Transport and lifecycle
- Tool design: names, descriptions, schemas, granularity
- Error handling & testing
- Security & deployment
- Demo: Running, testing & configuring MCP servers
- Exercise & PAIR Agent connection

Journey Checkpoint

Phase 1

- **Module 1:** Collaboration foundations
- **Module 2:** The WHAT — semantic context (AGENTS.md, ARCHITECTURE, etc.)
- **Module 3:** The HOW — procedural context, feedback loops, workflow stages

Phase 2

- **Module 4:** Zoomed in on process via **skills**
- **Module 5 (today): Tools** your assistant can invoke
- **Module 6:** The PAIR Agent — assembling skills + tools around your codebase

Learning Objectives

- **Goal:** Build, configure, and use an MCP server tailored to your codebase
- **Takeaway:** A working `domain-lang-mcp` your PAIR Agent uses in M6
- **Hope:** You leave with a *template* for building MCP servers, not just the one we wrote

Along the way you will:

1. Appreciate MCP among the broader tool-calling options
2. Design tools an LLM can use well (names, descriptions, schemas, errors)
3. Debug with MCP Inspector and register your server in your coding agent

Tool Calling Options

- Direct CLI access (you give the agent shell access)
- Code mode capabilities (built into your harness)
- **Model Context Protocol (MCP)** — today's topic
- Sub-agents-as-tools - covered in Phase 3

MCP wins where you need a **stable, discoverable, shareable** interface.

Why Model Context Protocol?

- **Standardised interface:** one protocol, not dozens of custom API shapes
- **Discoverable:** servers declare their tools/resources, no docs scraping
- **Scoped & safe:** servers expose only what's allowed (sandboxed, read-only, etc.)
- **Context-aware:** shared session state across calls
- **AI-native:** JSON-RPC + structured schemas easy for models to parse and call

But couldn't we just give the agent the CLI?

Yep, fair question.

The case for MCP over raw CLI access:

- **Stability:** today the agent calls `knowledge_graph lookup ...`. Tomorrow the backend moves to a hosted service. With MCP, the agent's call site doesn't change.
- **Sharability:** the same MCP can serve your pair agent *and* a business chatbot.
- **Decoupling:** the consumer (agent) and the implementation (CLI/API/DB) evolve independently.
- **Scoping:** you advertise only the operations you want exposed.

 For example: **domain-lang-mcp** (our exercise)

Today: stdio MCP → wraps the **knowledge-graph** CLI → SQLite.

Tomorrow: same MCP, HTTP transport → wraps a service backed by Neo4j.

From the agent's perspective: identical.

The Components of MCP

- **Host:** the application talking to the LLM (your IDE / CLI / harness)
- **Client:** the component inside the host that talks to MCP servers
- **Server:** the CLI tool or HTTP server that fulfills requests
- **Tool:** the named operation the LLM can invoke
 - Small, well-defined scope
 - May or may not use an LLM itself

Protocol: **JSON-RPC** messages over a stateful connection.

The Message Flow

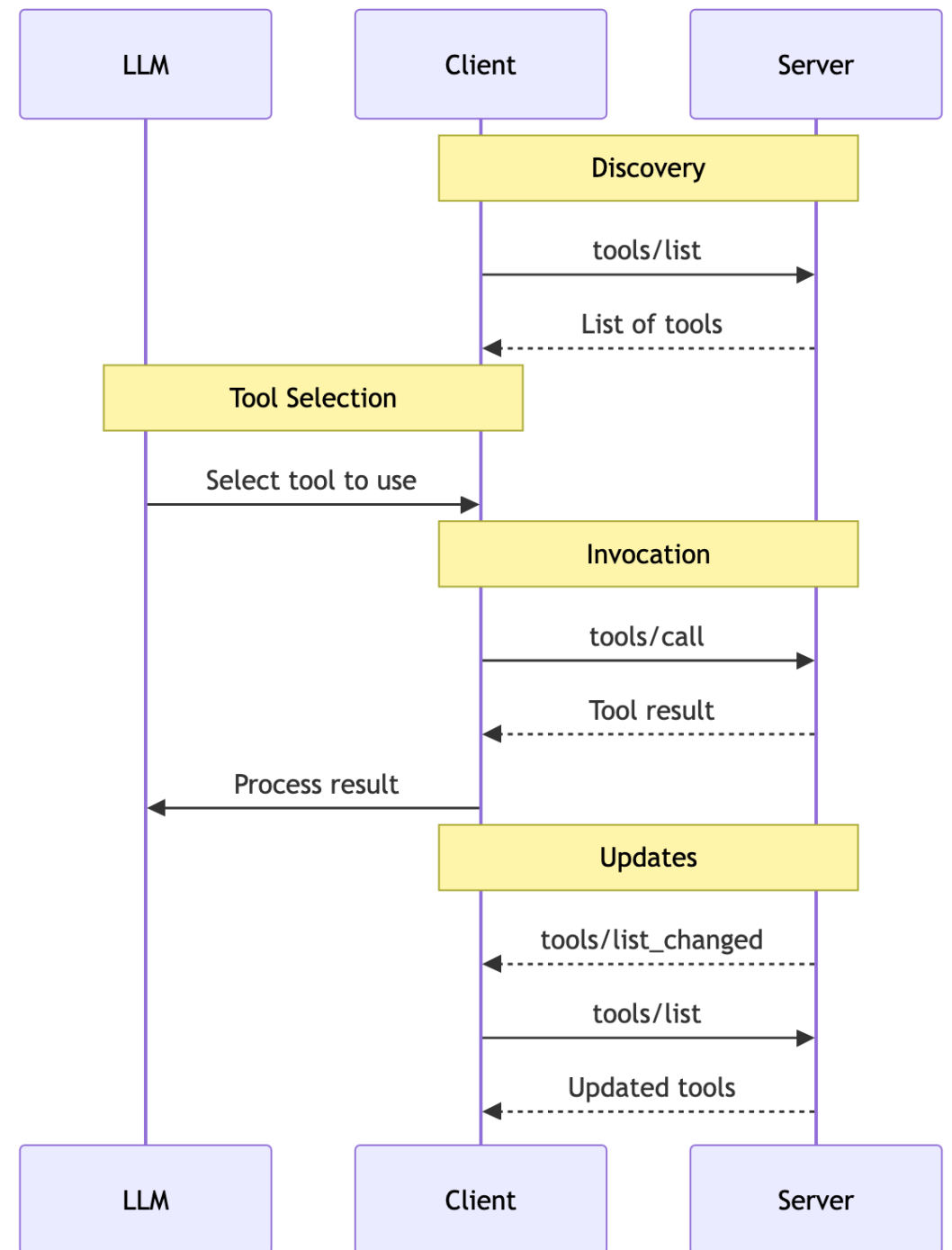
Four phases over a stateful connection:

Discovery — `tools/list`

Tool Selection — the LLM picks

Invocation — `tools/call` → result

Updates — `tools/list_changed`
keeps the client current



Capabilities

Server

- Tools: functions the agent can invoke
- Prompts: prepared prompts for repeating tasks
- Resources: static documents and files

Client

- Elicitation: server requests for additional information
- Sampling: server requests for recursive LLM interaction
- Roots: filesystem paths the client and LLM have access to

Transport Options

~~Three~~ Two official transports:

1. **stdio**: subprocess; JSON-RPC over stdin/stdout
 - Local, single client; process lifecycle tied to client
 - Lower latency; inherits parent permissions
2. **Streamable HTTP**: HTTP POST + GET
 - Multi-client; remote; scalable
3. ~~Server-Sent Events~~ — deprecated (lifecycle, recovery, scalability)

Match the transport to the **deployment**.

Initialisation & Lifecycle

1. Client → Server: `initialize` (capabilities, protocol version)
2. Server → Client: `initialize` response (advertises supported features)
3. Client → Server: `initialized` notification
4. Ready: serve `tools/list`, `tools/call`, etc.
5. Shutdown: release resources cleanly

Capability negotiation prevents runtime errors from unsupported features.



Wrap, Don't Rewrite

✗ "Do I need to re-implement my service as an MCP server?"

✓ No. Your MCP server is a bridge between AI and your existing:

- CLI tools
- REST APIs
- Databases
- Internal services

`domain-lang-mcp` wraps the `knowledge-graph` CLI/REST API.



Anatomy of a Tool

```
{
  "name": "lookup_term",
  "description": "Look up the definition, aliases, warnings, and source-file references for a domain term in the knowledge graph. Pass the term, its alias, or its display name.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "term": {
        "type": "string",
        "description": "Identifier, name, or alias of a domain term"
      }
    },
    "required": ["term"]
  }
}
```

The **description** is read by the LLM — it's the activation signal.

Tool Names and Descriptions

Verb-noun: `lookup_term`, `get_related_terms`, `list_domain_areas`.

✗ `query`, `execute`, `get_user_information_from_database`.

Descriptions include:

- *What* it does (action/purpose)
- *Inputs* expected (even though the schema shows this)
- *Output* shape
- *When* to use it (vs adjacent tools)

The LLM reads the description to decide *whether* to invoke. Optimise accordingly.

Schemas & Granularity

Schemas: use JSON Schema features — enums, patterns, min/max, descriptions for every field, required vs optional, sensible defaults.

Granularity: prefer fine-grained ... maybe

-  `create_user`, `update_user`, `delete_user`, `get_user`
-  `manage_user` with an `action` parameter

The LLM may pick better between many narrow tools than one wide tool with a complex parameter structure, but this has *changed* over time as models evolve.

Error Handling Categories

1. **Protocol errors** (MCP level): invalid method, malformed request
2. **Validation errors** (schema/business): missing required, wrong type, business rule violation
3. **Service errors** (underlying system): backend down, timeout, auth failure

Distinguish them so the LLM can react appropriately.

Testing Your Server

- **MCP Inspector** — interactive UI; lists tools; invokes manually; shows responses
 - Has a CLI mode for scripted tests
 - Essential during development
- **In-memory `Client`** (SDK) — fast unit tests; no subprocess; no network
- **Manual tests** with your agent, the actual *user*

Security & Deployment

Threats

- **Prompt injection** via tool output (separate research from execution)
- **Tool poisoning** via metadata changes (trust the source today *and* over time)
- **Lack of oversight** (broad permissions widen the blast radius)

Where MCPs run

- "On the metal" — `uv` / `npm` direct dependency
- Docker containers — stdio or HTTP
- VS Code marketplace, ChatGPT / Codex / Gemini marketplaces

Demo

Live: building, debugging, and registering `domain-lang-mcp`.

Watch for:

1. **Wrap, don't rewrite** — the CLI is the deterministic interface
2. **The description as activation signal** — the LLM's view of the tool
3. **MCP Inspector for debugging** — verify here before wiring into the harness

Exercise

Build your own **stdio MCP server** wrapping the provided `knowledge-graph` CLI.

You'll:

1. Design before you build (fill in `MCP_SERVER_DESIGN_TEMPLATE.md`)
2. Scaffold a stdio server in the language of your choice
3. Implement `ping` , `lookup_term` , `get_related_terms` , `list_domain_areas` , `validate_knowledge_graph`
4. Configure it in your coding agent

Optional stretch: build the HTTP variant wrapping the REST API.

Phase 2: PAIR Agent Connection

What you build today, your **PAIR Agent** uses in **Module 6**.

- M4 skill → M5 MCP → M6 PAIR Agent
- Your agent gains a *tool* it can reach for during real work
- Same `config-service` running thread — concrete, comparable, improvable